

ISIT307 - WEB SERVER PROGRAMMING

LECTURE 4 – ARRAYS



LECTURE PLAN

- Indexed arrays
- Associative arrays
- Manipulating arrays
- Sort, combine arrays
- Multidimensional arrays
- Use arrays in Web forms

INDEXED ARRAYS

- Syntax for declaring array:

```
$my_array = array(item1, item2, item3);
```

```
$my_array = [item1, item2, item3];
```

```
$my_array[] = item1;
```

```
$my_array[] = item2;
```

```
$my_array[] = item3;
```

MANIPULATING ARRAYS

- `array_push()` - adds one or more elements to the end of an array
 - it has variadic arguments – multiple elements can be added
 - returns the total number of the array elements
- `array_pop()` - removes the last element from the end of an array
 - returns the value of the removed array element
 - can be used with associative arrays as well
- `array_shift()` - removes the first element from the beginning of an array
 - returns the value of the removed array element
- `array_unshift()` - adds one or more elements to the beginning of an array
 - it has variadic arguments – multiple elements can be added

MANIPULATING ARRAYS

```
$indA1 = array("item1", "item2", "item3");

$num = array_push($indA3, 101, 102);

echo "<h2>Array 3 after push</h2>";
echo "<pre>";
print_r($indA3);
echo "</pre>";
echo"<p> indA3 now has $num elements</p>";

$elem = array_pop($indA3);

echo "<h2>Array 3 after pop</h2>";
echo "<pre>";
print_r($indA3);
echo "</pre>";
echo"<p>deleted element had value $elem</p>";

array_unshift($indA3, "B1", "B2");

echo "<h2>Array 3 after unshift</h2>";
echo "<pre>";
print_r($indA3);
echo "</pre>";

$elem = array_shift($indA3);

echo "<h2>Array 3 after shift</h2>";
echo "<pre>";print_r($indA3);
echo "</pre>";
echo"<p>deleted element had value
    $elem</p>";
```

MANIPULATING ARRAYS

- `array_splice()` - adds or removes array elements located anywhere in the array
 - rennumbers the indexes in the array

```
array_splice(array_name, start,  
            number_to_delete, values_to_insert);
```

MANIPULATING ARRAYS

- `unset()` - removes array elements and other variables
 - can remove multiple elements
 - do not renumber the next elements in the array
- `array_unique()` - removes duplicate elements from an array
 - returns a new array (with removed duplicate values), but does not renumber the indexes
- `array_values()` - can be used to renumber the indexed array elements
 - returns new array with renumbered indexes

ASSOCIATIVE ARRAYS

- Associative arrays use strings as indexes (*keys*)
- The element's key are specified by using the array operator (`=>`)

```
$my_array = array(key=>value, ...);
```

```
---
```

```
$my_array = [key=>value, ...];
```

```
---
```

```
$my_array[key1] = item1;
```

```
$my_array[key2] = item2; ...
```


ASSOCIATIVE ARRAYS

- If a new element is added to an associative array without specifying a key, the new element is assigned an index of 0 or the next available integer

```
$states["NSW"] = "New South Wales";  
$states["WA"] = "Western Australia";  
$states[] = "Tasmania";
```

- An array can have starting index other than 0 without creating empty elements

```
$states[100] = "New South Wales";  
$states[] = "Western Australia";  
$states[] = "Tasmania";
```

ITERATING ARRAYS

- The **internal array pointer** refers to the currently selected element in an array
- A **foreach** statement allows looping through the elements of an array, but it does not change the position of the internal array pointer - for that purpose advanced foreach should be used

```
foreach ($my_array as $key => $value)
{
    echo "The $key has $value";
}
```

MANIPULATING ARRAYS

- `in_array()` - determines whether a given value exists in an array and returns a Boolean value of true/false
- `array_search()` - determines whether a given value exists in an array and returns the index/key of the first matching element or return false if the value does not exist
- `array_key_exists()` - determines whether a given index or key exists and returns TRUE or FALSE
- `array_keys()` - returns an indexed array that contains all the keys in an associative array
 - If a second argument is passed, the function will return the keys of the elements that match the specified value

• For home - check the functions: `array_key_first()`, `array_key_last()`

MANIPULATING ARRAYS

- `array_slice()` - returns a portion of an array

```
array_slice(array_name, start, numbers_to_return);
```

```
---
```

```
$ThreeStates = array_slice ($states, 2, 3);  
foreach ($ThreeStates as $code => $state)  
{  
    echo "The code for $state is $code <br />";  
}
```

SORTING ARRAYS

- `sort()` - sorts an indexed array in ascending order
- `rsort()` - sorts an indexed array in descending order
- `asort()` - sorts an associative array in ascending order
- `arsort()` - sorts an associative array in descending order
- `krsort()` - sorts an array by keys in ascending order
- `krsort()` - sorts an array by keys in descending order
- `natsort()` - sorts an array using a "natural order"

COMBINING ARRAYS

- `array_combine()` - creates a new associative array by using the elements from one array as the keys and the elements from another array as the values
- array unpacking inside arrays - merging multiple arrays

```
$array1 = [1, 2, 3];  
$array2 = [4, 5, 6];  
$combined = [...$array1, ...$array2];
```

EXAMPLE – MESSAGE BOARD

TWO-DIMENSIONAL ARRAYS

- A **multidimensional array** consists of multiple indexes or keys
- A **two-dimensional** array has two sets of indexes or keys

TWO-DIMENSIONAL ARRAYS

```
$Ounces = array(1, 0.125, 0.0625, 0.03125, 0.0078125);  
$Cups = array(8, 1, 0.5, 0.25, 0.0625);  
$Pints = array(16, 2, 1, 0.5, 0.125);  
$Quarts = array(32, 4, 2, 1, 0.25);  
$Gallons = array(128, 16, 8, 4, 1);  
$VolumeConversions = array($Ounces, $Cups, $Pints,  
                             $Quarts, $Gallons);
```

	Ounces	Cups	Pints	Quarts	Gallons
Ounces	1	0.125	0.0625	0.03125	0.0078125
Cups	8	1	0.5	0.25	0.0625
Pints	16	2	1	0.5	0.125
Quarts	32	4	2	1	0.25
Gallons	128	16	8	4	1

TWO-DIMENSIONAL ARRAYS

```
$Ounces = array("ounces" => 1, "cups" => 0.125,  
               "pints" => 0.0625, "quarts" => 0.03125,  
               "gallons" => 0.0078125);  
$Cups = array("ounces" => 8, "cups" => 1,  
             "pints" => 0.5, "quarts" => 0.25,  
             "gallons" => 0.0625);  
$Pints = array("ounces" => 16, "cups" => 2,  
              "pints" => 1, "quarts" => 0.5,  
              "gallons" => 0.125);  
$Quarts = array("ounces" => 32, "cups" => 4,  
               "pints" => 2, "quarts" => 1,  
               "gallons" => 0.25);  
$Gallons = array("ounces" => 128, "cups" => 16,  
                "pints" => 8, "quarts" => 4, "gallons" => 1);
```

TWO-DIMENSIONAL ARRAYS

```
$VolumeConversions = array("Ounces" => $Ounces,  
    "Cups" => $Cups, "Pints" => $Pints,  
    "Quarts" => $Quarts, "Gallons" => $Gallons);
```

The diagram illustrates a two-dimensional array structure. A table contains conversion values between units. Above the table, the word 'Keys' has a downward arrow pointing to the first column. To the right of the table, the word 'Keys' has a leftward arrow pointing to the first row. A bracket on the right side of the table groups the rows and is labeled 'Elements'. A bracket below the table groups the columns and is labeled 'Elements'.

	"Ounces"	"Cups"	"Pints"	"Quarts"	"Gallons"
"Ounces"	1	0.125	0.0625	0.03125	0.0078125
"Cups"	8	1	0.5	0.25	0.0625
"Pints"	16	2	1	0.5	0.125
"Quarts"	32	4	2	1	0.25
"Gallons"	128	16	8	4	1

TWO-DIMENSIONAL ARRAYS

```
$VolumeConversions = array(  
    array(1, 0.125, 0.0625, 0.03125, 0.0078125), // Ounces  
    array(8, 1, 0.5, 0.25, 0.0625), // Cups  
    array(16, 2, 1, 0.5, 0.125), // Pints  
    array(32, 4, 2, 1, 0.25), // Quarts  
    array(128, 16, 8, 4, 1) // Gallons  
);
```

OR

TWO-DIMENSIONAL ARRAYS

```
$VolumeConversions = array(
    "ounces" => array("ounces" => 1, "cups" => 0.125,
        "pints" => 0.0625, "quarts" => 0.03125,
        "gallons" => 0.0078125),
    "cups" => array("ounces" => 8, "cups" => 1, "pints" => 0.5,
        "quarts" => 0.25, "gallons" => 0.0625),
    "pints" => array("ounces" => 16, "cups" => 2, "pints" => 1,
        "quarts" => 0.5, "gallons" => 0.125),
    "quarts" => array("ounces" => 32, "cups" => 4, "pints" => 2,
        "quarts" => 1, "gallons" => 0.25),
    "gallons" => array("ounces" => 128, "cups" => 16,
        "pints" => 8, "quarts" => 4, "gallons" => 1));
```

TWO-DIMENSIONAL ARRAYS

```
$users = [ ['name' => 'Tom', 'email' => 'tom@email.com'],  
           ['name' => 'Sam', 'email' => 'sam@email.com'],  
           ['name' => 'Kim', 'email' => 'kim@email.com']];
```

```
foreach($users as $user){  
    echo"<p>" . $user['name'] . "-" . $user['email'] . "</p>"; }  
-----
```

```
foreach($users as $user){  
    foreach ($user as $key => $info)  
        echo"$key: $info, ";}
```

```
-----  
foreach($users as ['name' => $name, 'email' => $email]){  
    echo"name: $name, email: $email <br>";}
```

USING ARRAYS IN WEB FORMS

- PHP arrays can be used with HTML form input elements - enabling to store the form data in an array
- The name attribute of the input element needs to use array notation
 - the data from any element with the same value for the name attribute will be appended to an array with that name
 - the indexes of the array elements can be used if there is a need, otherwise the only need is to append an opening and closing square bracket ([]) to the value of the name attribute

`name='req[]' or name='req[0]' or name='req[Q1]'`

USING ARRAYS IN WEB FORMS

EXAMPLE

```
<form action='ProcessForm.php' method='post' >
<p>Enter the first answer: <input type='text' name='answers[]' /></p>
<p>Enter the second answer:<input type='text' name='answers[]' /></p>
<p>Enter the third answer:<input type='text' name='answers[]' /></p>
<input type='submit' name='submit' value='submit' />
</form>
```

```
---
if (is_array($_POST['answers'])) {
    $Index = 0;
    foreach ($_POST['answers'] as $Answer) {
        ++$Index;
        echo "The answer for question $Index is '$Answer' <br />\n";
    }
}
```


USING ARRAYS IN WEB FORMS

EXAMPLE – QUIZ CAPITALS

Quiz - Capitals

The capital of NSW is:

The capital of Victoria is:

The capital of West Australia is:

The capital of Northern Territory is:

The capital of South Australia is:

The capital of ACT is:

The capital of Tasmania is:

Quiz - Capitals

Correct! The capital of NSW is Sydney.

Correct! The capital of Victoria is Melbourne.

Sorry, the capital of West Australia is not 'Per'.

Correct! The capital of Northern Territory is Darwin.

Sorry, the capital of South Australia is not 'South'.

Sorry, the capital of ACT is not 'Canberra'.

Correct! The capital of Tasmania is Hobart.

[Try again?](#)

ARRAYS – MORE EXAMPLES

- Example 1
- Example 2

